Memoria del proyecto · Actividad 3

Proyecto: Taskflow — gestión visual de tareas multi-organización

Tecnología: Ionic, Angular 22, TypeScript, Supabase y Vercel

Fecha de corte: 31 de octubre de 2026

Estado: Multi-organización con roles, administración de equipos, actividad por alcance, calendario, informes PDF y optimización SEO implementados; compilación, lint y pruebas automatizadas correctos.

1. Resumen ejecutivo

Sobre la base de la Actividad 2 (autenticación, CRUD de tareas, actividad, gráficos y tema oscuro) se ha convertido Taskflow en una aplicación **multi-organización** con tres roles (`SUPER_ADMIN`, `CORP_ADMIN` y `USER`). Se añadieron fecha límite y calendario, exportación de informes en PDF, gestión de equipos, asignación de tareas, un panel de super administrador, una página de actividad completa filtrada por rol, y mejoras de SEO, rendimiento y despliegue.

Validación local de este corte: compilación de producción correcta, lint correcto y **69 pruebas automatizadas aprobadas** (14 archivos). Lighthouse local sobre `/login` con el build de producción: **rendimiento 100, accesibilidad 100, buenas prácticas 100 y SEO 100**. Estos resultados validan el código local; no sustituyen una medición sobre el sitio desplegado.

2. Cambios desde la Actividad 2

Área	Cambio
Datos	Migraciones de `due_date`, Realtime de `activity_logs`, organizaciones, perfiles y RPC de administración
Roles	`SUPER_ADMIN`, `CORP_ADMIN`, `USER` con guards de ruta (`roleActivateGuard`, `roleMatchGuard`)
Registro	Sincronización `auth.users` → `profiles` mediante trigger; corrección del rol por defecto
Equipo	Alta, baja e invitación de usuarios; asignación y desasignación de tareas
Super admin	Estadísticas, edición y eliminación de organizaciones, y listado de miembros
Actividad	Resumen reciente en el panel y página completa con alcance por rol
Calendario	Vista mensual de tareas por fecha límite
Informes	Exportación PDF con `jspdf` y `jspdf-autotable`
Navegación	Rutas bajo `/workspace` y `/admin`; páginas de prohibido y no encontrado
SEO y despliegue	Metaetiquetas, `robots.txt`, source maps, `vercel.json` con reescritura SPA y cabeceras
Documentación	`README.md` con funcionamiento, pruebas, construcción y verificación

3. Arquitectura

- **Rutas** (`app.routes.ts`): `/login` y `/register` comparten `AuthPage`; `/workspace` (roles `USER` y `CORP_ADMIN`) contiene panel, `calendar`, `activity-feed` y `team-management` (solo `CORP_ADMIN`); `/admin` (solo `SUPER_ADMIN`) carga `ADMIN_ROUTES`. `/home` y `/calendar` redirigen a `/workspace`. Todas las páginas se cargan de forma diferida.
- **Servicios clave:** `SupabaseClientService` (cliente único, configuración en `assets/supabase-config.json`), `AuthService`, `ProfileService` (perfil, rol, invitaciones y RPC), `OrganizationService`, `TaskDataService` (tareas, asignación, actividad y Realtime) y `ReportExportService` (PDF).
- **Componentes compartidos:** tarjetas de estadística y de tarea, tabla de datos con selector de responsable y gráficos del panel.

- **Se eliminó** la pestaña «Editor completo» y su guard de cambios pendientes, al no aportar valor.

4. Base de datos y seguridad

Migraciones en `supabase/migrations/`, que deben ejecutarse en orden en el SQL Editor de Supabase:

1. `20261027170000_create_task_tables.sql` — tablas, RLS, índices y Realtime.
2. `20261027175000_restrict_activity_log_inserts.sql` — auditoría por trigger.
3. `20261029090000_add_task_due_date.sql` — fecha límite.
4. `20261029104500_enable_activity_logs_realtime.sql` — Realtime de actividad.
5. `20261031133000_create_organizations_and_admin_rpc.sql` — organizaciones y RPC de administración.
6. `20261031144000_sync_auth_users_to_profiles.sql` — sincronización de usuarios con `profiles`.
7. `20261031160000_organization_admin_features.sql` — funciones auxiliares, políticas RLS, RPC de gestión de usuarios y asignación, triggers e invitaciones.

La autorización no depende solo del cliente: los roles se aplican con RLS y RPC en Supabase, y las rutas con guards. No se guarda ninguna clave `service_role` en el frontend. Tras un cambio de rol en `app_metadata`, el usuario debe volver a iniciar sesión.

5. Funcionalidades por rol

Usuario (`USER`*)*

- Gestiona sus tareas, ve el calendario y exporta informes.
- En actividad solo ve eventos de **sus** tareas.

Administrador de organización (`CORP_ADMIN`*)*

- Desde el Cuadro de Mando y la página **Gestión de equipo** añade y elimina usuarios de su organización y asigna o desasigna tareas.
- Ve toda la actividad de su organización.
- La página de equipo incluye un botón **Volver al inicio**.

Super administrador (`SUPER_ADMIN`*)*

- Panel con número de organizaciones y de usuarios por organización.
- Edita y elimina organizaciones y consulta sus miembros.

6. Errores corregidos

1. **Creación de corporación:** la restricción `check_valid_role` rechazaba el rol por defecto `STANDARD_USER`; se corrigió el valor por defecto en la migración de sincronización.
2. **Registro:** se eliminó un validador asíncrono que llamaba a un RPC `check_email_exists` inexistente; se añadió `auth.page.spec.ts`.
3. **Navegación del panel:** con `<base href="/">`, los enlaces `href="#..."` navegaban a `/` y «Tareas» llevaba a login. Se sustituyeron por desplazamiento programático y `routerLink`.
4. **SEO 45 %:** se añadieron metadatos, `robots.txt` y source maps, y una reescritura SPA para evitar 404 en rutas profundas en Vercel.

7. SEO, rendimiento y despliegue

- `src/index.html`: `description`, `robots` y etiquetas Open Graph.
- `src/robots.txt`: permite el sitio y bloquea `/workspace` y `/admin`.
- `angular.json`: `robots.txt` como asset y source maps de scripts en producción.
- `vercel.json`: carpeta `www`, reescritura SPA, cabeceras `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy`, `Cross-Origin-Opener-Policy`, una CSP mínima (`frame-ancestors`, `object-src`, `base-uri`) y caché inmutable para `.js` y `.css` con hash.

Una CSP completa y Trusted Types no se activaron para no romper el script inline que genera Angular; en Lighthouse son informativos.

8. Validación de este corte

Comprobación	Resultado
`npm test -- --watch=false`	Correcto: 14 archivos y 69 pruebas aprobadas
`npm run build`	Correcto: bundle de producción en `www`, con source maps y `robots.txt`
`npm run lint`	Correcto: todos los archivos pasan
Lighthouse local (`/login`, producción)	Rendimiento 100 · Accesibilidad 100 · Buenas prácticas 100 · SEO 100

Las migraciones 5 a 7 se probaron en un PostgreSQL local (PGlite) durante el desarrollo; las pruebas automatizadas no ejecutan acciones contra la instancia real de Supabase.

9. Límites conocidos y siguientes pasos

- Volver a desplegar en Vercel y medir Lighthouse sobre la URL real; el 45 % original se midió allí y la causa exacta es una hipótesis.
- Verificar que `assets/supabase-config.json` se publica en cada entorno.
- Probar el flujo con varias cuentas y organizaciones reales para confirmar las políticas RLS remotas.
- Lighthouse aún avisa de JavaScript sin usar (`chart.js` y `jspdf` podrían cargarse bajo demanda), caché y reflow forzado; no restan puntuación.
- Algunos estilos de componente (`super-panel`, `team-management`, `calendar`) están cerca del presupuesto de 4 kB.
- `TaskService` y `TaskDataService` mantienen responsabilidades cercanas; conviene consolidarlos en un cambio separado.

10. Archivos de referencia

- `src/app/app.routes.ts`: rutas y guards por rol.
- `src/app/home/home.page.ts`: panel, equipo, asignación y navegación.
- `src/app/features/workspace/team-management/`: gestión de equipo.
- `src/app/features/workspace/activity-feed/`: actividad completa por alcance.
- `src/app/features/admin/super-panel/`: panel de super administrador.
- `src/app/core/services/{profile,organization,report-export}.service.ts` y `src/app/core/tasks/task-data.service.ts`: servicios.
- `supabase/migrations/20261031160000_organization_admin_features.sql`: núcleo de la administración.
- `vercel.json`, `src/robots.txt`, `src/index.html`: SEO y despliegue.

- `README.md`: guía de uso, pruebas y verificación.